Univerza v Ljubljani
Fakulteta za računalništvo in informatiko

Klemen Remec

Emulacija računalnika Sinclar ZX Spectrum

DIPLOMSKO DELO

UNIVERZITETNI ŠTUDIJSKI PROGRAM
PRVE STOPNJE
RAČUNALNIŠTVO IN INFORMATIKA

Mentor: izr. prof. dr. Jurij Mihelič

Ljubljana, 2026

To delo je ponujeno pod licenco *Creative Commons Priznanje avtorstva-Deljenje pod enakimi pogoji 2.5 Slovenija* (ali novejšo različico). To pomeni, da se tako besedilo, slike, grafi in druge sestavine dela kot tudi rezultati diplomskega dela lahko prosto distribuirajo, reproducirajo, uporabljajo, priobčujejo javnosti in predelujejo, pod pogojem, da se jasno in vidno navede avtorja in naslov tega dela in da se v primeru spremembe, preoblikovanja ali uporabe tega dela v svojem delu, lahko distribuira predelava le pod licenco, ki je enaka tej. Podrobnosti licence so dostopne na spletni strani creativecommons.si ali na Inštitutu za intelektualno lastnino, Streliška 1, 1000 Ljubljana.

Izvorna koda diplomskega dela, njeni rezultati in v ta namen razvita programska oprema je ponujena pod licenco GNU General Public License, različica 3 (ali novejša). To pomeni, da se lahko prosto distribuira in/ali predeluje pod njenimi pogoji. Podrobnosti licence so dostopne na spletni strani `http://www.gnu.org/licenses/`.

Besedilo je oblikovano z urejevalnikom besedil LaTeX.

Kandidat: Klemen Remec
Naslov: Emulacija računalnika Sinclair ZX Spectrum z vpogledom v stanje sistema
Vrsta naloge: Diplomska naloga na univerzitetnem programu prve stopnje Računalništvo in informatika
Mentor: izr. prof. dr. Jurij Mihelič

Opis:
Besedilo teme diplomskega dela študent prepiše iz študijskega informacijskega sistema, kamor ga je vnesel mentor. V nekaj stavkih bo opisal, kaj pričakuje od kandidatovega diplomskega dela. Kaj so cilji, kakšne metode naj uporabi, morda bo zapisal tudi ključno literaturo.

Title: Emulation of Sinclar ZX Spectrum computer with system state insight

Description:
opis diplome v angleščini

Na tem mestu zapišite, komu se zahvaljujete za pomoč pri izdelavi diplomske naloge oziroma pri vašem študiju nasploh. Pazite, da ne boste koga pozabili. Utegnil vam bo zameriti. Temu se da izogniti tako, da celotno zahvalo izpustite.

Kazalo

Povzetek

Abstract

1 Uvod **1**

2 ZX Spectrum **3**

2.1 Zgodovina podjetja Sinclair in računalnikov ZX Spectrum . . . 3

2.2 Pregled strojne opreme 4

2.3 Pregled emulacije in simulacije 6

2.4 Programiranje emulatorja in razhroščevalnika 6

3 Pomnilnik **7**

3.1 Implementacija v emulatorju 9

4 Centralna procesna enota Z80 **11**

4.1 Registri in zastavice 12

4.2 Ukazi 16

4.3 Prekinitve 21

4.4 Implementacija v emulatorju 23

5 ULA **25**

5.1 Časovnik 26

5.2 Zaslon 26

5.3 Tipkovnica 30

5.4 Kasete 35
5.5 Zvok 36
5.6 Implementacija v emulatorju 37

6 Evalvacija emulatorja **39**

7 Zaključek **41**

Literatura **43**

Seznam uporabljenih kratic

kratica	angleško	slovensko
CA	classification accuracy	klasifikacijska točnost
DBMS	database management system	sistem za upravljanje podatkovnih baz
SVM	support vector machine	metoda podpornih vektorjev

Povzetek

Naslov: Emulacija računalnika Sinclar ZX Spectrum

Avtor: Klemen Remec

V vzorcu je predstavljen postopek priprave diplomskega dela z uporabo okolja LaTeX. Vaš povzetek mora sicer vsebovati približno 100 besed, ta tukaj je odločno prekratek. Dober povzetek vključuje: (1) kratek opis obravnavanega problema, (2) kratek opis vašega pristopa za reševanje tega problema in (3) (najbolj uspešen) rezultat ali prispevek diplomske naloge.

Ključne besede: emulacija, Sinclair ZX Spectrum, digitalno ohranjanje.

Abstract

Title: Sinclar ZX Spectrum emulation

Author: Klemen Remec

This sample document presents an approach to typesetting your BSc thesis using LaTeX. A proper abstract should contain around 100 words which makes this one way too short.

Keywords: emulation, Sinclair ZX Spectrum, digital perseverance.

Poglavje 1

Uvod

Kratek opis računalnika

Kratka zgodovina

Napoved teme in (konkretnih) ciljev diplomskega dela

Hiter pregled vsebine po poglavjih

Poglavje 2

ZX Spectrum

2.1 Zgodovina podjetja Sinclair in računalnikov ZX Spectrum

Modeli računalnika ZX Spectrum so se skozi čas spreminjali, in ena od glavnih sprememb so bile velikosti pomnilnika. Osnovni model ima oznako 16K (tako imenovano zaradi 16kB velikega pomnilniškega prostora), kasneje pa sta izšla še modela ZX Spectrum+ z oznako 48K (najbolj poznan in razširjen model), in ZX Spectrum 128 z oznako 128K. **V tem delu se bomo nanašali na model 48K.**

Razvoj najbolj znanih programov in razširjenosti po svetu

Slovenski vpliv na ZX Spectrum, Jakopin

Pomen ohranjanja računalniških sistemov in programske opreme preko emulacije

2.2 Pregled strojne opreme

Kratek pregled strojne opreme

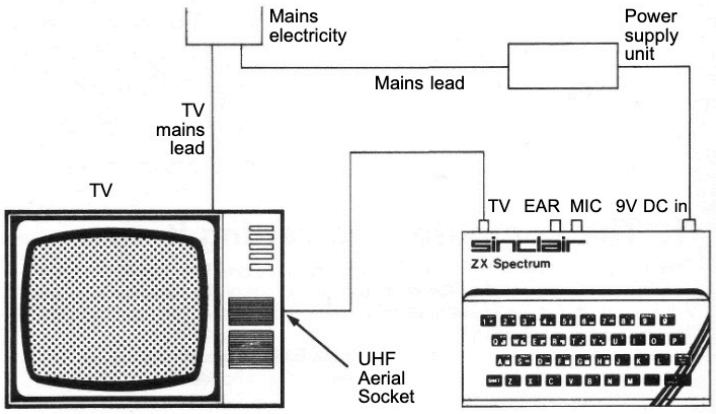
Računalnik ZX Spectrum ima na zadnji strani več vtičnic oz. priključkov: vtičnico 9V DC IN, ki se preko napajalne enote priključi v električno omrežje, 3,5 milimeterski vtičnici EAR, ki se lahko poveže z izhodno vtičnico za slušalke ali zvočnik na kasetofonu, in MIC, ki se poveže z vhodno vtičnico za mikrofon na kasetofonu, ter vtičnico za televizijo in robni priključek za povezavo z dodatno opremo. [3]

Ko računalnik ugasnemo, se vse v njem shranjene informacije izgubijo, če jih pred tem ne prenesemo npr. na kaseto. Če se kadarkoli zapletemo pri uporabi računalnika, ga lahko le izključimo iz električnega omrežja in nazaj vklopimo, ter tako ponastavimo brez skrbi poškodbe računalniku. [3]

CPE Z80 predstavlja možgane računalnika, ki nadzira vse operacije, izvaja aritmetične račune, preverja pritisnjene tipke, sprejema odločitve in na splošno upravlja, kaj naj računalnik naredi. Sam ne ve nič o BASIC-u, temveč prejema vsa navodila iz ROM-a. [3]

ROM (Read Only Memory) vsebuje 16384 (16K) navodil v strojnem kodu Z80, ki sestavljajo osnovni program računalnika, in procesorju narekuje, kaj naj stori v vseh predvidljivih okoliščinah. Ta program je edinstven za ZX Spectrum. [3]

RAM (Random Access Memory) obsega osem čipov za shranjevanje podatkov, ki jih procesor želi obdržati, kot so programi v BASIC-u, spremen-



Slika 2.1: Skica povezav komponent sistema. [3]

ljivke, slika za televizijski zaslon in različni elementi, ki spremljajo stanje računalnika. [3]

ULA (Uncommitted Logic Array) deluje kot komunikacijski center sistema, ki poskrbi, da se vse, kar zahteva procesor, tudi dejansko izvede, hkrati pa bere pomnilnik za sestavo televizijske slike in pošilja ustrezne signale televizijskemu vmesniku. [3]

PAL kodirnik je skupina komponent, ki pretvarja televizijski izhod iz logičnega čipa v obliko, primerno za barvne televizije. [3]

Regulator napetosti pretvarja rahlo nestabilno napetost napajanja v popolnoma konstantnih pet voltov za zanesljivo delovanje sistema. [3]

UHF/VHF modulator prenaša sliko na televizijski zaslon. [3]

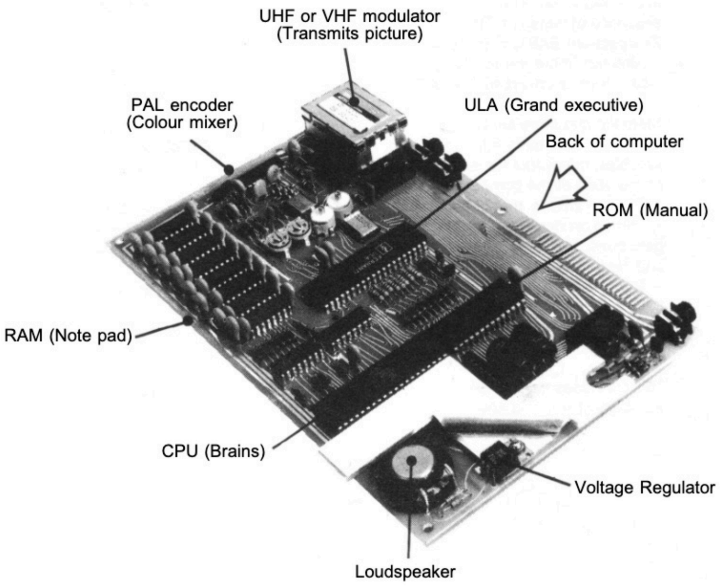
Zvočnik predstavlja zvočni izhod za zvok. [3]

Ostala dodatna oprema

Obstaja še druga oprema, ki jo boste lahko uporabimo s Spectrumom. [2]

ZX Microdrive je naprava za hitro množično shranjevanje in je veliko bolj prilagodljiva pri načinu uporabe kot kasetni snemalnik. [2]

Omrežje se uporablja za povezovanje več računalnikov ZX Spectrum, da lahko med seboj komunicirajo. [2]



Slika 2.2: Skica notranjih komponent računalnika ZX Spectrum. [3]

Vmesnik RS232 omogoča standardno povezavo računalnika s tipkovnicami, tiskalniki, računalniki in drugimi napravami, tudi če niso bili zasnovani posebej za Spectrum. [2]

2.3 Pregled emulacije in simulacije

Kratek pregled že obstoječih emulatorjev

2.4 Programiranje emulatorja in razhroščevalnika

Uporabljene tehnologije

Kratek pregled delovanja emulacije in vpogleda v sistem

Poglavje 3

Pomnilnik

V pomnilniku se hrani podatke v obliki zlogov (vredosti med 0 in 255), ki se nahajajo na dvozložnih naslovih med 0 in FFFFh. [2]

Razdelitev delov pomnilnika in njihov pomen

V računalniku ZX Spectrum, je pomnilnik razdeljen na ROM (bralni pomnilnik), ki se nanaša na naslovni prostor med naslovoma 0x0000 in 0x3FFF, ter RAM (bralno-pisalni pomnilnik), ki se nanaša na naslovni prostor med naslovoma 0x4000 in 0xFFFF. [2]

Branje vrednosti iz pomnilnika deluje tako pri delu z ROMom in RAMom, pisanje vrednosti v pomnilnik pa deluje le v RAMu (ukaz pisanja v ROM se bo izvedel, a brez učinka). [1]

RAM je nadalje razdeljen na sekcije z različnimi nameni uporabe, pomembnejše sekcije so sledeče:

- zaslonska datoteka vsebuje podatke za prikaz vsebine na zaslon
- atributi položajev znakov so shranjeni v formatu ATTR, in so v pomnilniku shranjeni zaporedno po vrsticah
- medpomnilnik tiskalnika, velik 256 zlogov med pomnilniškima nasovoma 0x5B64 in 0x5BFF, hrani znake, ki jih mora tiskalnik natisniti,

kadar pa tiskalnika ne uporabljamo, ostaja ta prostor neizkoriščen [1]

- sistemske spremenljivke vsebujejo razne informacije o stanju računalnika
- sistem BASIC, ki vsebuje uporabniški program v BASICu, spremeljivke in druge podatke ter sklad za aritmetični kalkulator, in se razteza do naslova, na katerega kaže sistemska spremenljivka RAMTOP
- prosti pomnilnik je zavarovan pred vplivi programa v BASIC-u in pred posegi nadzornega programa. [1]
- strojni sklad vsebuje sklad, ki ga uporablja procesor Z80. V Spectrumu se sklad začenja 2 zloga pod vrhom neporabljenega naključnega pomnilnika. [1]
- računski sklad predstavlja 30 zlogov velik prostor z začetkom na naslovu, shranjenem v sistemski spremenljivki MEMBOT, ki se uporablja za računske operacije v plavajoči vejici
- definicije uporabniško definiranih grafičnih znakov

[2]

Pri vnosu več podatkov v dano sekcijo, kot je privzeta velikost dane sekcije, se ostale sekcije premaknejo navzgor, in obratno pri brisanju podatkov. [2]

ROM

V ROM-u shranjen nadzorni program poleg pravilnega zagona računalniškega sistema opravlja tudi mnoge druge naloge, kot so čakanje na pritiske tipk, izrisujovanje slike na zaslon, tolmačenje pognanega programa v BASICu in posredovanje napak o njegovem izvajanju, delo s plavajočo vejico, itd. [1]

Neposredno izvajanje strojnega programa po drugi strani deluje mimo nadzornega programa. [1]

Sistemske spremenljivke

Zlogi na pomnilniških naslovih med 23552 in 23733 se uporabljajo za hrambo in spreminjanje informacij o delovanju sistema. [2]

Bolj pomembne sistemske spremenljivke so sledeče:

- VARS na naslovu 0x5C4B, ki vsebuje naslov začetka spremenljivk programa BASIC
- PROG na naslovu 0x5C53, ki vsebuje naslov začetka programa v BASICu
- E_LINE na naslovu 0x5C59, ki vsebuje naslov konca programa v BASICu
- RAMTOP na naslovu 0x5CB2, ki vsebuje naslov zadnjega zloga sistema BASIC
- P-RAMT na naslovu 0x5CB4, ki vsebuje naslov zadnjega zloga fizičnega pomnilnika
- BORDCR na naslovu 0x5C48, katerih prvi trije biti definirajo barvo obrobe zaslona
- ATTR_P na naslovu 0x5C8D, ki vsebuje definicije uporabljanih barv papirja in papirja, ter drugih atributov stalnega prikaza znakov
- CHARS na naslovu 0x5C36, ki vsebuje naslov na začetek nabora znakov

[2] [1]

Str. 127-130 - tabela sistemskih spremenljivk, njihovih pomenov in navodil uporabe [2]

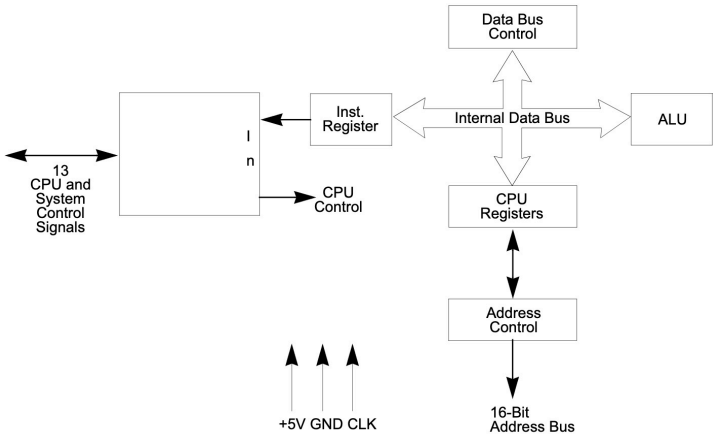
3.1 Implementacija v emulatorju

Poglavje 4

Centralna procesna enota Z80

Procesor Z80 najdemo v mnogih računalnikih - Spectrum, ZX 81, Galaksija, Dialog 20 Gorenje, Amstrad CPC 454 in celo večji računalniki, kot je Iskrin Partner. [1]

Družina komponent CPE Z80 podjetja Zilog je četrta generacija izboljšanih mikroprocesorjev, ki dosega hitrosti od 6 do 20 MHz. [4] Ima mnogo 8-bitnih in nekaj 16-bitnih registrov, a ker velja za 8-bitni procesor, večino časa obdeluje 8-bitne podatke v 8-bitnih registrih in 8-bitnih pomnilniških celicah. [1]



Slika 4.1: Diagram notranje arhitekture in glavnih elementov procesorja Z80 [4].

Vsak sistem s procesorjem Z80 mora vsebovati določene strojne elemente, kot so vir napajanja +5V, oscilator, pomnilniške naprave (RAM, ROM ali PROM) in vhodno-izhodna vezja, katerih signali so časovno usklajeni s procesorjem za nadzor pomnilnika ali perifernih vezij. [4]

4.1 Registri in zastavice

Razporeditev registrov, njihove velikosti in pomen

Večina operacij Z80 je 8-bitnih, torej dela z vrednostmi med 0x00 in 0xFF, kar lahko predstavlja bodisi nepredznačen obseg vrednosti od 0 do 255, bodisi predznačen obseg vrednosti od -128 do 127. [1]

Nekatere operacije pa so 16-bitne, torej CPE dela z vrednosti med 0x0000 in 0xFFFF, kar lahko predstavlja bodisi nepredznačen obseg vrednosti od 0 do 65535, bodisi predznačen obseg vrednosti od -32768 do 32767. 16-bitna števila največkrat potrebujemo za naslavljanje pomnilniških celic. [1]

Posebnost so vrednosti števil v obliki plavajoče vejice, ki zavzemajo kar 5 zlogov (prvi zlog vsebuje $128+n$, ostali 4 zlogi vsebujejo mantiso z vrednostjo $[1/2, 1)$). Ker 5-zložnih vrednosti ne moremo shraniti v posamezen register ali registrski par, za njihovo shranjevanje uporabljamo bodisi celoten registrski petorček AFBCDE, bodisi računski sklad v pomnilniku. [1]

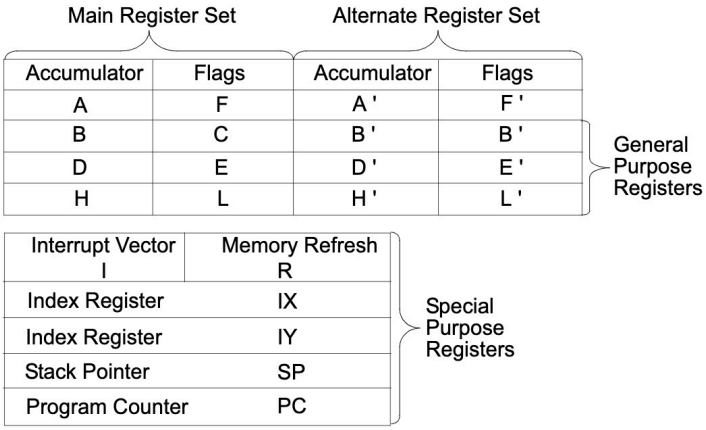
Procesor Z80 vsebuje 208 bitov bralno-pisalnega pomnilnika, implementiranega kot statični RAM, ki predstavlja 18 8-bitnih registrov in 4 16-bitne registre. [4]

Ti notranji registri se delijo na dva registrska nabora, od katerih vsak vsebuje 6 splošno-namenskih registrov (registrih B, C, D, H in L), akumulator (register A) in register zastavic (register F), ter 6 posebno-namenskih registrov, ki so 16-bitni programski števec (register PC), 16-bitni kazalec sklada (register SP), 16-bitna indeksna registra IX in IY, osvežitveni register R in register prekinitev I. [1] [4]

Pri tem lahko 8-bitne registre uporabljamo posamično, ali pa par 8-bitnih registrov združimo v 16-bitni registrski par (v katerem prvi register vsebuje

višji del vrednosti, drugi pa nižjega). V procesorju Z80 tako lahko uporabljamo registrske pare BC, DE in HL. [1] [4]

Med osnovnim in zamenljivim registrskim naborom (keterega registre označujemo z apostrofom ob imenu) se lahko menjava, kar pride prav tako za ohranjanje stanja vrednosti registskega nabora med izvajanjem podprograma, ki bi moral sicer prejšnje registrske vrednosti shraniti v pomnilnik in jih pred vrnitvijo obnoviti. Ta značilnost procesorja Z80 prav tako pride prav v sistemih, ki zahtevajo hiter odziv na prekinitve, saj je mogoče en niz registrov splošnega namena in akumulator ter register zastavic rezervirati za obdelavo prekinitve brez potrebe po shranjevanju in kasnejšem pridobivanju vsebine registrov v pomnilniku med obdelavo prekinitev ali podprogramov. [1] [4]



Slika 4.2: Diagram notranjih registrov procesorja Z80 [4].

Akumulator oz. register A velja za edini register, kjer procesor lahko opravlja 8-bitne aritmetično-logične operacije. [1] Pri tem register zastavic F v splošnem hrani posebne pogoje izvedenih operacij ali dobljenega rezultata. [4]

Register naslova strani prekinitve (I) hrani 8 najvišjih bitov posrednega naslova, medtem ko naprava, ki povzroči prekinitev, zagotovi osem najnižjih bitov naslova prekinitveno-servisnega programa. Procesor Z80 namreč deluje

v načinu, v katerem je mogoče v odziv na prekinitev izvesti posredni klic na PSP na poljubnem mestu v pomnilniku z minimalnim časom dostopa do rutine. [4]

Register osveževanja pomnilnika (R) hrani 7-bitni števec osveževanja pomnilnika, ki se samodejno poveča po vsakem pridobljenem ukazu, osmi bit se uporablja le pri ukazu 'LD R, A'. To omogoča enako enostavno uporabo dinamičnih pomnilnikov kot statičnih pomnilnikov brez upočasnjevanja delovanja CPE (osveževanje se dogaja med dekodiranjem prej pridobljenega ukaza). Med osveževanjem se vsebina registra I shrani na zgornjih osmih bitih naslovnega vodila. [4]

Za 16-bitne registre velja, da je višji zlog shranjen v registru/registrskem paru prvi, nižji pa drugi. Pri nalaganju 16-bitnih števil v pomnilnik velja obraten dogovor kot pri uporabi registrskih parov - nizki zlog je v pomnilniku zmeraj spravljen prvi. [1]

Indeksna registra (IX in IY) hranita 16-bitni naslov, ki se uporablja v indeksiranih načinih naslavljanja kot **bazni kazalec**, iz katerega po prištetju odmika (dodaten zlog, predstavljen kot število v dvojiškem komplementu), dobimo končni pomnilniški naslov, iz katerega se podatki shranjujejo ali pridobivajo. Ta način naslavljanja med drugim močno poenostavi uporabo tabel podatkov v pomnilniku. [4]

Procesor ima poseben 16-bitni register, ki ga imenujemo programski števec PC (angl. Program Counter), ki hrani naslov naslednjega ukaza za izvršitev. PC se uporablja tudi za klicanje in vračanje iz podprogramov, pri čemer se povratni naslov hrani na skladu v pomnilniku. [1] Programski števec (PC) hrani 16-bitni naslov trenutnega ukaza, ki se pridobiva iz pomnilnika, in se samodejno poveča, ko se njegova vsebina prenese na naslovno vodilo. [4]

Naslov vrhnje vrednosti na skladu je shranjen v posebnem 16-bitnem registru SP (angl. Stack Pointer). [1] Kazalec sklada (SP) vsebuje 16-bitni naslov trenutnega vrha sklada, ki se nahaja kjerkoli v pomnilniku. Sklad je organiziran kot LIFO (angl. Last-In-First-Out). Sklad omogoča preprosto izvajanje večstopenjskih prekinitev, neomejeno gnezdenje podprogramov in

poenostavitev mnogih vrst manipulacije podatkov. [4]

Zastavice

Zastavice, hranjene v registru F (in F'), predstavljajo informacijo o stanju bodisi vsebine registra A, ali o ravnokar opravljeni računski operaciji. Kljub temu, da ima register F možnost shranjevanja 8 bitov, za zastavice uporablja le 6 bitov (bita 3 in 5 se ne uporabljata):

- Bit 7 hrani zastavico znaka S (angl. Sign flag): dvignjena, ko je rezultat operacije negativen
- Bit 6 hrani ničelno zastavico Z (angl. Zero flag): dvignjena, ko je rezultat operacije enak nič.
- Bit 4 hrani zastavico polprenosa H (angl. Half carry flag): dvignjena, ko se je v zadnji operaciji spremenil 5. bit registra
- Bit 3 hrani zastavico parnosti/prepolnjenja P/V (angl. Parity/oVerflow flag): dvignjena, ko je v rezultatu operacije parno število bitov, oz. ko se je spremenil 8. bit registra (kar pomeni možnost preliva)
- Bit 2 hrani zastavico odštevanja N (angl. subtract flag): dvignjena, ko je zadnja operacija bila odštevanje
- Bit 1 hrani zastavico prenosa C (angl. Carry flag): dvignjena, ko se je v zadnji operaciji zgodil prenos

[1] [4]

Str. 65 - tabela bitnih lokacij zastavic v registru zastavic [4]

Zastavice so v procesorju edine, ki omogočajo vejanje programa preko pogojnih skokov z možnimi pogoji:

- dvignjena ničelna zastavica Z (angl. Zero)
- spuščena ničelna zastavica NZ (angl. Not Zero)

- dvignjena zastavica prenosa C (angl. Carry)
- spuščena zastavica prenosa NC (angl. Not Carry)
- dvignjena zastavica znaka M (angl. Minus)
- spuščena zastavica znaka P (angl. Positive)
- dvignjena zastavica parnosti PE (angl. Parity Even)
- spuščena zastavica znaka PO (angl. Parity Odd)

[1]

Nedokumentirane zastavice

4.2 Ukazi

ZX Spectrum omogoča uporabo visokonivojskega jezika, imenovanega BASIC (Beginners All-purpose Symbolic Instruction Code). Jezik je bil leta 1964 razvit na Dartmouth College v New Hampshiru, in je bil v svojem času zelo razširjen na osebnih računalnikih. Ima več različic, med katerimi obstajajo manjše razlike, kot na primer zahteva po zapisu ukazne besede LET pri dodeljevanju vrednosti spremenljivkam, kar v mnogih drugih različicah jezika BASIC lahko opustimo. A ZX Spectrum BASIC vseeno ni daleč od (neobstoječih) standardov jezika BASIC, zato ne bi smeli imeti preveč težav pri prilagajanju katerega koli BASIC programa za delovanje na ZX Spectrum. [3]

Za programiranje na Spectrumu lahko uporabljamo tudi strojni jezik, ki omogoča hitrejše izvajanje, učinkovitejšo izrabo pomnilnik, krajše programe (za hrambo v pomnilniku) ter neodvisnost od nadzornega programa. A prinaša tudi nekatere slabosti, kot so slabša berljivost programov, strojni programi so uporabni le za izvajanje v sistemu z istim procesorjem, ter nasploh daljši programi (večje število ukazov). [1]

Berljivost strojnega programa lahko izboljšamo s pisanjem v zbirnem jeziku, ki predstavlja preslikavo strojnega jezika v ljudem razumljivejšo obliko, v kateri vsak zbirni ukaz ustreza enemu strojnemu ukazu (in obratno), in lahko v ukazih naslov nadomestimo z labelo danega naslova. Zbirni jezik je tako razumljivejši ljudem, a ga mora računalnik (natančneje program zbirnik) pred izvajanjem prevesti v zaporedje zlogov strojnega koda, bodisi ročno, bodisi z zbirnikom (najbolj razširjen je bil npr. GENS). [2] [1]

Število ukazov, ki jih je mogoče med programiranjem shraniti v pomnilniku, je omejeno. ZX Spectrum to omejitev označi z zvočnim signalom. [3]

Število ukazov in sklopi, njihove dolžine

Procesor Z80 lahko izvaja 158 različnih vrst ukazov, vključno z vsemi 78 ukazi procesorja 8080A. [4]

APPENDIX A: Nabor ukazov [2]

Ukaze delimo na več skupin:

- ukaze nalaganja in izmenjav, ki premikajo podatke med izvorno in ciljno lokacijo, bodisi med posameznimi registri, ali registri in pomnilnikom

- aritmetične in logične ukaze, ki jih procesor izvaja nad podatki v akumulatorju in drugih splošnonamenskih registrih ali pomnilniških lokacijah. Rezultati danih operacij se nato shranijo v akumulatorju, glede na rezultat operacije pa se nastavijo tudi ustrezne zastavice v registru zastavic.

- ukaze rotacije in zamikov, ki omogočajo levo ali desno, aritmetično ali logično, rotacijo ali zamik vrednosti poljubnega registra ali pomnilniške lokacije z ali brez prenosa

- ukaze manipulacije bitov, ki omogočajo nastavitev, ponastavitev ali preveritev vrednosti poljubnega bita v akumulatorju, drugem splošnonamenskem registru ali pomnilniški lokaciji. Ta skupina je še posebej uporabna v

nadzornih aplikacijah in za nadzorovanje programskih zastavic v programiranju splošnega namena.

- ukaze skokov, klicev in vrnitev, ki se uporabljajo za skoke med več lokacijami v programu. Edinstvena vrsta klica je navodilo RST, ki lahko kot nov naslov izvajanja določi le enega od 8 ločenih naslovov, ki se nahajajo na prvi strani pomnilnika.

- ukaze prenosov in iskanja blokov, ki se uporabljajo za prenos celih blokov podatkov iz enega mesta pomnilnika na drugo oziroma iskanje poljubnega 8-bitnega znaka v bloku podatkov v pomnilniku. Ob dosegu konca definiranega bloka ali najdenega znaka se ukaz samodejno prekine, prav tako se lahko izvajanje takega ukaza kadarkoli prekine in kasneje nadaljuje, kar omogoča vmesno uporabo procesorja za druge operacije. Ukazi te skupine pridejo posebaj prav pri pri obdelavi velikih nizov podatkov.

- ukaze vhoda in izhoda, ki omogočajo prenose podatkov (posameznih ali blokov) med pomnilniškimi lokacijami ali splošnimi registri procesorja in zunanjimi V/I napravami. Številka V/I vrat se pri teh operacijah pojavi na spodnjih 8 bitih naslovnega vodila.

- nadzorni oziroma kontrolni ukazi, ki omogočajo upravljanje različnih načinov delovanja procesorja, med drugim omogočanje in onemogočanje prekinitev, nastavljanje načina odziva na prekinitve, itd.

[4]

Skupina ukazov RST (angl. ReSTart) opravlja isto nalogo kot brezpogojni klici, le da lahko z njimi kličemo le 8 naslovov v začetku ROM-a:

- RST 0x00: učinkuje kot izklop in ponovnen vklop računalnika

- RST 0x08: služi za sporočila o napakah v BASIC-u, slediti mora 8-bitni kod napake

- RST 0x10: na zaslon izpiše znak, katerega kod je v registru A

- RST 0x18: register A napolni z vrednostjo zloga, katerega naslov je v sistemski spremenljivki CH-ADD
- RST 0x20: se uporablja pri tolmačenju programov v BASIC-u
- RST 0x28: se uporablja za računanje s plavajočo vejico, sledijo zlogi, ki določajo operacijo
- RST 0x30: preuredi delovni prostro tako, da dobimo v njem toliko prostih zlogov, kolikor je vrednost registrskega para BC
- RST 0x38: preleti tipkovnico (kontrola pritisnjenih tipk) in poveča števec časa.

[1]

Uporabniki Sinclairjevega vmesnika 1 lahko z ukazom RST 08H upravljajo mikrotračne enote, vmesnik RS232 in prenos podatkov v omrežju Spectrumov. [1]

VIR: I. Logan: Spectrum microdrive book (Melbourne House Publishers, Tring 1983) [1]

Ukaz RST 28H v Spectrumu označuje, da želimo izvesti operacije (1 ali več) na računskem skladu, operacijo definiranmo z nizom kodov operacij, ki se mora končati z vrednostjo 38H (označuje, da sledi strojni ukaz). [1]

Str. 132-133 - kodi operacij za ukaz RST 28 in tabele podprogramov [1]

Tipi naslavljanj

Naslavljanje se nanaša na način definicije naslova podatkov (v registrih, pomnilniku ali V/I vratih), ki se bodo uporabili pri izvedbi določenega ukaza. [4]

Registrsko naslavljanje (oblike ‘LD r, r‘) določi en ali več registrov, ki se uporabijo za dano operacijo. [1] [4]

Sprotno ali takojšnje naslavljanje (oblike ‘LD r, n‘) v zlogu, ki sledi št. strojnega ukaza, vsebuje 8-bitno vrednost operanda. *Str. 34 - diagram sestave ukaza* [1] [4]

Razširjeno takojšnje naslavljanje (oblike ‘LD rr, nn‘) v zlogih, ki sledita št. strojnega ukaza, vsebuje 16-bitno vrednost operanda. *Str. 34 - diagram sestave ukaza* [4]

Zunanje ali razširjeno naslavljanje (oblik ‘LD A, (nn)‘ in ‘LD (nn), A)‘) v zlogih, ki sledita št. strojnega ukaza, vsebuje pomnilniški naslov bodisi izvorne skočne lokacije ali (izvora ali cilja) operanda (kar omogoča prenašanje vrednosti iz registra v pomnilniško celico in obratno, skok ali prenos podatkov med poljubnima pomnilniškima lokacijama) prenašanje vrednosti iz registra v pomnilniško celico, in obratno. *Str. 36 - diagram sestave ukaza* [1] [4] lokacijama.

Registrsko posredno naslavljanje (oblik ‘LD r, (rr)‘ in ‘LD (rr), r‘) določi registrski par, katerega vrednost se uporabi kot polnilniški naslov. [1]

Relativno naslavljanje v zlogu, ki sledi št. strojnega ukaza, vsebuje odmik (dvojiški komplent), ki se prišteje trenutni lokaciji izvajanja (vrednosti registra PC), na katerega naj se izvede skok. To poleg prenosljivosti programa omogoča tudi skoke na bližje pomnilniške lokacije z le dvozložnim ukazom. *Str. 35 - diagram sestave ukaza* [4]

Registrsko posredno naslavljanje določi registrski par, katerega vrednost predstavlja pomnilniškegi naslov operanda. *Str. 37 - diagram sestave ukaza* [4]

Indeksno naslavljanje (oblik ‘LD r, (IX+dis)‘ in ‘LD (IX+dis), r‘) je le oblika registskega posrednega naslavljanja z dodanim zamikom - v zlogu, ki sledi št. strojnega ukaza, vsebuje odmik v predstavitvi dvojiškega komplementa, ki se prišteje izbranemu indeksnemu registru za določitev pomnilniške lokacije operanda. To poleg prenosljivosti programa omogoča tudi poenostavitev dela s tabelami podatkov - prenosljivost programa. *Str. 36 - diagram sestave ukaza* [1] [4]

Bitno naslavljanje s pomočjo registrskega, registskega posrednega ali in-

deksnega naslavljanja lahko določi poljubni bit poljubne pomnilniška lokacije ali registra za izvedbo bitne operacije. [4]

Spremenjeno naslavljanje ničelne strani se nanaša na posebne enozložne ukaze RST za skok na poljubno od 8 lokacij v strani 0 pomnilnika, kjer se nahajajo pogosto uporabljeni podprogrami ROM-a. *Str. 35 - diagram sestave ukaza* [4]

Implicitno naslavljanje se uporablja pri mnogih ukazih, ki implicitno določijo registre z operandi, potrebnimi za dano operacijo (npr. rezultat aritmetičnih operacij se implicitno shrani v akumulator). [4]

Ker mnoge operacije zahtevajo več kot en operand, se lahko hkrati uporabi tudi več tipov naslavljanj. [4]

Nedokumentirani ukazi

4.3 Prekinitve

Prekinitve omogočajo zunanjim napravam prekinitev operacij CPE, izvajanja PSP in vrnitev v prvotno stanje izvajanje programa. [4]

Na primer, CPE vsako 1/50 sekunde pogleda stanje posebne notranje zastavice IFF1 (angl. Interrupt Flip Flop 1). Če je njena vrednost 1, prekine svoje delo in izvrši ukaz RST 38H, s katerim pridobi stanje tipkovnice. S tem se delovna hitrost seveda nekoliko zmanjša, zato kdaj želimo prekinitve onemogočiti, in kasneje nazaj omogočiti. [1]

Prav tako lahko programer z ukazom HALT prekine izvajanje strojnega programa do naslednje prekinitve ob omogočenem IFF. Med tem CPE izvaja operacije NOP za namen delovanja osveževanja pomnilnika. [1] [4]

VIR: G. Smiljanič: Mikroračunala (Školska knjiga, Zagreb 1983) [1]

Načini prekinitev in njihovo delovanje

Z80 CPE podpira 2 vrsti prekinitev:

Prekinitve INT, ki jih je mogoče maskirati (programsko onemogočiti) z

nastavljanjem vrednosti IFF (Interrupt enable Flip-Flop) preko ukazov EI in DI (prekinitve so omogočene oz. onemogočene šele po izvedbi naslednjega ukaza od EI/DI zaradi izvedbe ukaza vrnitve iz PSP), saj se uporablja kot zahteva za prekinitev s strani zunanjih naprav. [4]

CPE ima pravzaprav IFF1 (dejansko uporabljen pri izvajanju) in IFF2 (začasna shramba vrednosti IFF1). *Str. 18, 19 - tabela stanj in sprememb stanj IFF1 in IFF2* [4]

CPE je lahko nastavljen v enega izmed 3 možnih načinov obravnavanja **maskable** prekinitev:

- Način 0: prekinitvena naprava lahko na podatkovno vodilo postavi poljuben ukaz (pogosto klic nadaljnjega PSP, saj mora izbran ukaz imeti le 1 zlog), ki ga CPE izvede. Število urinih period, potrebnih za izvedbo tega ukaza, je za dva večje od običajnega za izbran ukaz, saj CPE samodejno doda 2 čakalni periodi, da bi zagotovil dovolj časa za izvedbo zunanje verižne povezave za nadzor prioritet.

- Način 1: CPE začne izvajati ukaz na pomnilniškem naslovu 0038h. Posledično je odziv identičen odzivu na **nonmaskable** prekinitev, razen da je lokacija klica 0038h namesto 0066h. Število urinih period, potrebnih za izvedbo tega ukaza, je za dva večje od običajnega za izbran ukaz, saj CPE samodejno doda 2 čakalni periodi.

- Način 2: predstavlja najmočnejši način odzivov na prekinitve, saj omogoča klic podprograma na poljubnem mestu v pomnilniku. Za uporabo tega načina programer vzdržuje tabelo 16-bitnih začetnih naslovov PSP na poljubnem mestu v pomnilniku. 16-bitni naslov PSP se združi iz vrednosti registra I (zgornjih 8 bitov) in vrednosti dobljene iz prekinitvene naprave (spodnjih 8 bitov; bit 0 mora imeti vrednost 0, saj se vrednosti tabele nahajajo na sodih naslovih, ker vsak zavzema 2 zloga). Prvi zlog v tabeli je najmanj pomemben (najnižji del naslova). Ko prekinitvena naprava zagotovi spodnji del kazalca, CPE samodejno potisne PC na sklad, pridobi naslov PSP iz tabele in izvede skok nanj. Ta način odziva

zahteva 19 čakalnih urinih period.

[4]

Prekinitve NMI, ki jih ni mogoče maskirati (programsko onemogočiti) in imajo višjo prioriteto kot INT, saj se uporablja za predvsem za takojšnje odzive na pomembne signale. CPE ga obravnava po koncu izvajanja trenutnega ukaza:

1. IFF1 se onemogoči (in njegova prejšnja vrednost shrani v IFF2), kar prepreči nadaljnje navadne prekinitve, dokler jih program spet ne omogoči
2. PC shrani na sklad
3. izvajanje preusmeri na pomnilniški naslov 0066h

[4]

4.4 Implementacija v emulatorju

Poglavje 5

ULA

Procesorja pri branju in pisanju iz pomnilniških lokacij ne zanima s katerim delom pomnilnika dela, pa naj bo to ROM, RAM, ali celo neobstoječ del pomnilnika, ve le, da obstaja 65536 pomnilniških naslovov, na katerih lokacijah lahko obdeluje shranjene podatke. Na popolnoma enak način dela tudi s 65536 V/I naslovi, preko katerih komunicira z zunanjimi napravami (tipkovnica, zvočnik, barvo obrobe zaslona, kasetofon preko vtičnic EAR in MIC, tiskalnik in drugi robni vmesniki in naprave pa preko robnega priključka). [2] [1]

V/I naslovi je sestavljeni iz 16 bitov (A15, A14, ..., A2, A1, A0), ki imajo različne pomene - vrednost 0 katerikoli bita od A0, A1, A2, A3 ali A4 predstavlja indikator specifične operacije, bitov A6 in A7 računalnik ne uporablja, bita A8 in A9 včash podajata dodatne (nedokumentirane) informacije.

Ker v večini primerov višji zlog V/I naslova nima nobenega vpliva na delovanje, je odločilen le nižji zlog V/I naslova, ki označuje V/I vrata (različni V/I naslovi tako mnogokrat naslavljajo ista izmed 256 V/I vrat). [2] [1]

Naslov V/I vrat 254 se nanaša na zvočnik (D4), MIC vtičnico (D3) in nastavlja barvo obrobe zaslona (D2, D1 in D0). Naslov vrat 251 se nanaša na tiskalnik. Naslovi vrat 254, 247 in 239 se nanašajo na dodatne zunanje naprave [2]

Vrednosti na V/I naslovih, ki prehajajo med zunanjimi napravami preko

V/I vrat in CPE so sestavljena sestavljena iz 8 bitov [2] [1]

Kontrolno, podatkovno in naslovno vodilo so dostopna na zadnji strani računalnika. *Str. 118 - prikaz dostopnih povezav do vodil* [2]

5.1 Časovnik

Notranja ura ZX Spectruma je napram ustavljanju programa (2% natančnost) veliko bolj natančna (0,01% natančnost), a se začasno ustavi med spuščanjem zvoka ali operacijami s kasetofonom, tiskalnikom in drugimi zunanjimi napravami. [2]

Hrani se na pomnilniških pozicijah 23674, 23673 in 23672, ki se uporabljajo kot števec 1/50 oz. 1/60 sekunde (odvisno od frekvence osveževanja zaslona) sekunde. [2]

Čas ene urine periode (kremenčevega kristala v CPE) imenujemo stanje T (angl. Transition state), ki traja 0,2857 mikrosekunde, in predstavlja osnovno merilo hitrosti operacij. [1]

Dolžine ukazov in T-stanja

Vsi ukazi so zaporedje osnovnih operacij (strojnih M stanj), sinhronih na urine periode (časovna T stanja).

5.2 Zaslon

V času nastanka računalnika ZX Spectrum so se uporabljali UHF televizorji, kateremu je računalnik oddajal zaslonski signal na UHF kanalu 36. Pomembna je bila tudi različica računalnika, saj so se standardi televizorskih sistemov razlikovali po državah in celinah - evropski in britanski televizorji takrat prikazovali 50 slik na sekundo, sistemi v ZDA, Kanadi in Japonski pa 60 slik na sekundo. Zaradi tega so zanje delali drugačno verzijo računalnika. [3]

Velikost

Televizijski zaslon ima 24 vrstic, vsako dolgo 32 znakov. Vrstice zaslona so oštevilčene od 0 (vrh) do 21, stolpci pa od 0 (leva) do 31. [2]

Razdelitev

Zaslon je razdeljen na dva dela. Zunanji del se imenuje obroba zaslona (angl. border), osrednji del pa papir (angl. paper). Barve teh dveh delov je mogoče poljubno spreminjati, tako neposredno s tipkovnico kot tudi v programu. [3]

Zaslonska slika je razdeljena na 768 položajev (preko 24 vrstic in 32 stolpcev), na katerih se prikazujejo grafični simboli oziroma znaki. Vsak grafični simbol je predstavljen kot 8x8 kvadrat točk, od katerih je vsaka lahko obarvana bodisi v barvo papirja (angl. paper) če ima pripadajoč bit vrednost 0, bodisi v barvo črnila (angl. ink) z vrednostjo bita 1. Cela oblika simbola je tako predstavljena v 8 zlogih (vsak za eno vrstico točk). [2]

A ti niso shranjeni v zaporednih pomnilniških celicah, saj snop elektronov v televiziji potuje po potrebuje te zaporedne informacije posameznega snopa. V pomnilniku so v takem zaporedju vrstic pikslov shranjene najprej vrstice položajev znakov 0 do 7, nato 8 do 15, in nazadnje 16 do 23. [2]

Str. 78 - prikaz delitve zaslona na vrstice in stolpce [2]

Str. 114 - Diagram razbiranja naslovov PRINT položajev in njihove naslove prilaskov (barve papirja in črnila, svetlosti in utripanja) [1]

Nabor znakov

Nabor 256 znakov, ki ga uporablja ZX Spectrum, sestavljajo

- posamezne znake (npr. črke, številke, ločila ipd.)
- znake iz standarda ASCII (razen simbolov £ in ©), in imajo tudi iste kode
- znake, značilne le za ZX Spectrum, kot so grafični simboli (uporabljani

v grafičnem načinu delovanja tipkovnice), katerih je prvih 16 sistemsko definiranih, drugih 21 pa uporabniško-definiranih

- žetoni (angl. tokens), ki predstavljajo bodisi kontrolne znake, ki imajo učinek na zunanje naprave (npr. zaslon), bodisi cele besede (npr. PRINT, STOP, ipd.)

[2]

V ROM-u so med naslovoma 3D00 in 3FFF shranjeni točkovni vzorci vseh znakov razen grafičnih. Naslov posameznega znaka, ki ima kod X (npr. 'a' ima kod 61H) izračunamo po formuli $8x + 3C00$ [1]

Str. 68 - tabela grafičnih simbolov [2]

Str. 87 - tabela simbolov in načinov tipkovnice da prideš do njih [2]

APPENDIX A: Nabor znakov [2]

Običajno, ko je nekaj zapisano na položaj znaka, popolnoma uniči tisto, kar je bilo tam zapisano prej. Lahko pa spremembo naredimo tako, da bo novi znak preprosto dodan na vrhu starega. To je lahko še posebej koristno za pisanje sestavljenih znakov, kot so črke s poudarki na njih, kot v tem programu za tiskanje nemških črk - "ož umlaut nad njim. [2]

Atributi znakov

Vsak znak ima določene sledeče atribute v enem zlogu:

- bit, ki določa svetlost barv, ima dve možni vrednosti: 0 za normalne ali 1 za svetlejše barve svetlo.

- bit, ki določa utripanje, ima dve možni vrednosti: 0 za stabilne barve ali 1 za utripanje, ki se prikaže kot zaporedno zamenjavanje barve črnila in papirja.

- trojica bitov, katerih vrednost določa kodo barve papirja (privzeto bela)

- trojica bitov, katerih vrednost določa kodo barve črnila (privzeto črna)

[3] [2] [1]

ZX Spectrum lahko na (barvnem) zaslonu prikazuje osem barv, označene s številkami od 0 do 7:

- 0 - črna
- 1 - modra
- 2 - rdeča
- 3 - vijolična ali magenta
- 4 - zelena
- 5 - bledo modra ali cian
- 6 - rumena
- 7 - bela

[3] [2]

Čeprav so barve na prvi pogled razporejene v naključnem vrstnem redu, je tak zaradi dveh razlogov. Prvi razlog se nanaša na uporabo računanika na črno-belih zaslonih, kjer se barve v danem vrstnem redu prikažejo v padajočih odtenkih sivin. Drugi razlog se opira na dejstvo, da lahko človeško oko resnično vidi le tri primarne barve (modro, rdečo in zeleno), druge barve pa so mešanice le-teh. Tak vrstni red barv omogoča "mešanje"barv s seštevanjem kod primarnih barv - npr. magenta je narejena z mešanjem modre in rdeče barve, zato je njena koda (3) vsota kod za modro (1) in rdečo (2). [3] [2]

Kot barvo črnila ali papirja lahko nastavimo tudi

- vrednost 8, ki označuje "prozorno- barva, s katero jo uporabljate, bo ostala nespremenjena ob izpisu znaka
- vrednost 9, ki pomeni "kontrast- barva (črnilo ali papir), s katero jo uporabljate, bo nastavljena v kontrastu z drugo - bela, če je druga barva temna (črna, modra, rdeča ali magenta) ali črna, če je druga barva svetla (zelena, cianska, rumena ali bela)

[2]

Ker se atributi nanašajo na vseh 64 pik položaja znaka (ne pa na posamezne pike), ne moremo določiti več kot 2 barv ali nastavitev svetlosti in utripanja. Ko nekaj natisnemo na zaslon, poleg vzorca pik spremenimo tudi nastavitve atributov položaja. [2]

5.3 Tipkovnica

Tipkovnica Spectruma je zelo podobna tipkovnici navadnega pisalnega stroja - tipke črk in številk so na istih mestih, vendar pa ima vsaka tipka več funkcij. [3]

TODO: SLIKA RAZPOREDITVE TIPKOVNICE

Za popravljanje napak uporabimo tipko DELETE. V zgornji vrstici tipkovnice so štiri puščice, ki kažejo v različne smeri, in beseda DELETE. Za uporabo teh tipk morate pri pritisku uporabiti tipko CAPS SHIFT. Stranske puščice premikajo kurzor v levo ali desno, tipka DELETE pa izbriše znak neposredno pred kurzorjem. [3]

Ko pritisnete ENTER, bo računalnik vaše sporočilo zapisal na vrh zaslona - ali pod tisto, ki ga je zapisal prvič, če je še vedno tam. [3]

Če katero koli tipko pridržimo pritisnjeno več kot tri sekunde, se ta samodejno ponovi, tipka pa takrat oddaja tiho klikajoč zvok. [3] [2]

Znaki, ki jih ZX Spectrum prepozna in uporablja, ne vključujejo le posameznih simbolov (črk, številk itd.), ampak tudi sestavljene znake (ključne besede, imena funkcij itd.). Da bi dobili vse te funkcije in ukaze, imajo nekatere tipke pet ali več različnih pomenov, do katerih pridemo bodisi preko SHIFT tipk (CAPS SHIFT ali SYMBOL SHIFT) bodisi z menjavanjem načinov delovanja tipkovnice. [2]

Načini delovanja

Kurzor je predstavljen kot utripajoča črka, ki označuje tako položaj izpisa naslednjega vnešenega znaka kot tudi trenutni način delovanja tipkovnice

preko simbola kurzorja. [3] [2]

Ko Spectrum prvič vklopite, ta po sporočilu o avtorskih pravicah pričakuje vnos ukaza, ki se začne s ključno besedo, katere so natisnjene pod glavnimi simboli na tipkah. Za razliko od večine drugih računalnikov Spectrum omogoča vnos ključnih besed z enim samim pritiskom na tipko. Kurzor s simbolom K (angl. Keyword) predstavlja ta način delovanja za vnos ključnih besed, ki samodejno nadomesti način L, ko naprava pričakuje vnos ukaza ali programske vrstice, kar je odvisno od pozicije kurzorja v vrstici. [3] [2]

V večini drugih primerov bomo v računalnik želeli vnašati besedilo, kar naredimo v načinu delovanja za vnos navadnega besedila. V njem ima kurzor simbol L (angl. Letter). [3] [2]

Na načinu K in L velja, da je za vse kontrolne funkcije, ki so nad tipkami obarvani belo, potrebna pritisnjena tipka CAPS SHIFT, za vse pomožne znake, ki so pod tipkami obarvani rdeče, pa tipka SYMBOLS SHIFT. Če želimo vnesti veliko velikih črk, ne da bi držali tipko CAPS SHIFT, lahko vse črke vnašamo kot velike, tako da najprej pritisnemo tipko CAPS LOCK (CAPS SHIFT in 2). CAPS SHIFT z drugimi tipkami ne vpliva na ključne besede v načinu K, v načinu L pa pretvori male črke v velike. [3] [2]

Način delovanja za vnos besedila z velikimi črkami, v katerem ima kurzor simbol C (angl. Capitals), je različica načina L, v katerem se vse črke prikažejo kot velike. Pritisk na tipko CAPS LOCK povzroči prehod iz načina L v način C ali obratno. [3] [2]

Poleg ključnih besed in besedila (malih in velikih črk ter številk) ima tipkovnica tudi osem vnaprej definiranih grafičnih znakov, ki so na tipkovnici označeni na številčnih tipkah od 1 do 8, ali pa uporabniško-določene grafične znake. Ti se lahko natisnejo na zaslon na podoben način kot črke in številke. Za to je treba tipkovnico preklopiti v način delovanja za vnos grafičnih znakov, kar se naredi s pritiskom tipke GRAPHICS (kombinacija tipk CAPS SHIFT in 9). Ta način traja, dokler se ponovno ne pritisne bodisi tipka GRAPHICS ali pa le tipka 9. Kurzor bo v tem načinu delovanja označen s simbolom G (angl. Graphics) [3] [2]

Obstaja še en zadnji - razširjeni - način, ki ga označuje kurzor s simbolom E (angl. Extended). Do njega pridemo s hkratnim pritiskom tipk CAPS SHIFT in SYMBOL SHIFT, in traja le do vključno naslednjega pritiska tipke. Ta način omogoča uporabo večine znanstvenih in programskih funkcij, ter pridobitev dodatnih znakov oz. simbolov:

- zgoraj prikazan v zeleni barvi, če med tem ni pritisnjena tipka SHIFT
- spodaj prikazan v rdeči barvi, če je pritisnjena ena od tipk SHIFT
- simbol na številčni tipki, če je pritisnjena tipka SYMBOL SHIFT
- sicer da barvno kontrolno zaporedje

[3] [2]

Kurzor s simbolom vprašaja pomeni, da je pri vnosu ukaza ali znaka prišlo do napake. [3]

Branje stanja tipkovnice

Za branje tipkovnice lahko uporabimo podprograme v ROMu: [1]

KEY SCAN (naslov 0x02BE) vrne rezultat v registrih D in E:

- če ni pritisnjena nobena tipka, bo vrednost DE enaka FFFF
- če je pritisnjena le ena tipka, bo register D vseboval FF, register E pa bo vseboval kod pritisnjene tipke
- če sta pritisnjeni CAPS SHIFT ali SYMBOL SHIFT in katerakoli druga tipka, bo register D vseboval kod SHIFT tipke, register E pa kod druge pritisnjene tipke
- če sta pritisnjeni tipki CAPS SHIFT in SYMBOL SHIFT istočasno, bo vrednosti DE enaka 2718

in v ničelni zastavici: v zgornjih primerih bo zastavica dvignjena, če pa je naenkrat pritisnjenih več tipk, bo ničelna zastavica ob povratku spuščena, a pri tem spremeni vrednosti registrov A, B, C, D, E, H in L. [1]

Podprogram KEY TEST (naslov 031E) se uporablja za zavržbo rezultate KEY SCAN:

- FFFF (nobena pritisnjena tipka)
- FF27 (pritisnjena le tipka CAPS SHIFT)
- FF18 (pritisnjena le tipka SYMBOL SHIFT)

če je v DE šmiselna vrednost", bo podprogram najprej vrednost registra D bo prenesel v register B ter v register D naložil 0 ,in nadaljeval v eno od dveh smeri:

- če je v DE neveljavna vrednost, bo v register A naložil vrednost registra E (drugi zlog rezultata) in spustil zastavico prenosa
- če je v DE kakršnakoli druga vrednost, bo v register A naložil kod osnovnega znaka tipke in dvignil zastavico prenosa

[1]

(ASCII) kode osnovnih znakov tipk dobimo v BASICu z ukazom CODE, sicer pa v posebni tabeli KEY TABLE na naslovu 0205. [1]

Podprogram KEY CODE (naslov 0333) ugotovi, kateri znak (ne tipka) je bil uporabljen, in naloži njegov ASCII kod v register A:

- register E mora vsebovati kod osnovnega znaka
- register B mora vsebovati FF/27/18
- glede na način delovanja tipkovnice:
  - za način G mora register C vsebovati 02
  - za način E mora register C vsebovati 01

  - za način F mora register C vsebovati 00 in register D 00
  - za način L mora register C vsebovati 00, register D 08 in bit 3 sistemske spremenljivke FLAGS 2 mora biti 0
  - za način C mora register C vsebovati 00, register D 08 in bit 3 sistemske spremenljivke FLAGS 2 mora biti 1

- register C mora vsebovati vrednost sistemske spremenljivke MODE
- register D mora vsebovati vrednost sistemske spremenljivke FLAGS

[1]

Podatke iz tipkovnice, poleg z uporabo podprogramov v ROMu, lahko pridobimo tudi neposredno preko V/I vrat 0xFE. Tipkovnica je razdeljena v 8 pol-vrstic s 5 tipkami, podatke za pritisnjene tipke lahko zato pridobimo le v petericah tipk naenkrat. [1]

Str. 101 - Skica skupin tipk [1]

Skupino tipk pri tem določimo z vrednostjo registra A ob začetku izvrševanja ukaza (zaporedni bit, ki predstavlja številko željene skupine, mora imeti vrednost 0, vsi ostali pa 1). [1]

Naslov se izračuna kot $254 + 256 * (255 - 2^n)$; $n \in [0, 7]$:

- naslov 0xFEFE predstavlja tipke med tipkama CAPS SHIFT in V
- naslov 0xFDFE predstavlja tipke med tipkama A in G
- naslov 0xFBFE predstavlja tipke med tipkama Q in T
- naslov 0xF7FE predstavlja tipke med tipkama 1 in 5
- naslov 0xEFFE predstavlja tipke med tipkama O in 6
- naslov 0xDFFE predstavlja tipke med tipkama P in 7
- naslov 0xBFFE predstavlja tipke med tipkama ENTER in H
- naslov 0x7FFE predstavlja tipke med tipkama SPACE in B

[2] [1]

Register A po izvršenem ukazu branja stanja tipkovnice vsebuje 8-bitni zlog podatkov, pri čemer spodnjih pet bitov (od D0 do D4) označuje pritisnjeno tipko (najnižji bit ustreza najbolj zunanji tipki; za pritisnjeno tipko ima bit vrednost 0, sicer 1). Sicer hkrati lahko, s primerno nastavitvijo registra A pred izvedbo ukaza, beremo tudi več skupin tipk hkrati, a posamezen bit rezultata v tem primeru predstavlja zaporedno pritisnjeno tipko katerekoli izmed izbranih skupin tipk. Sedmi bit (D6) prebranega zloga pa predstavlja vrednost vtičnice EAR. [1] [2]

Str. 121 - Skica tipkovnice Spectruma in kodi tipk (razvrščeni v spirali v nasprotni smeri urinega kazalca) [1]

Ker je neposredno branje stanja tipk precej okorno, zato se marsikdaj programi raje poslužijo zgoraj omenjenega podprograma iz ROM-a. [1]

5.4 Kasete

Uporaba - branje in pisanje, preverjanje, itd

Namesto, da bi morali vsakič ponovno v računalnik vnašati programe, ZX Spectrum omogoča snemanje programov, spremenljivk in drugih oblik podatkov za ponovno uporabo na kasete z običajnim domačim kasetofonom. To ROM omogoča z ukazom SAVE, pravilnost prenešenih spremenljivk in programa na kaseti pa lahko nato preverimo pravilnost napram programu v računalniku z ukazom VERIFY. Kasneje lahko program iz kasete naložimo v pomnilnik z ukazom LOAD, in ga nato poženemo z ukazom RUN. [3] [2]

V vsakemu od teh ukazov lahko bodisi sledi niz, ki predstavlja ime programa (10 znakov, sestavljeno le iz črk in številk), ki ga želimo shraniti, preveriti, naložiti ali pognati. V primeru ukazov LOAD, VERIFY in RUN lahko niz pustimo tudi prazen, kar privzeto pomeni obdelavo prvega najdenega programa. [3] [2]

Formati

Ob nalaganju vidimo vzorce barvnih vodoravnih črt, ponavadi v podobnem vzorcu:

- 5 sekund rdečih in svetlo modrih črt, širokih približno 1 cm, ki se počasi premikajo navzgor
- zelo kratek izbruh modrih in rumenih črt
- 1 sekunda, ko je vse normalno
- 2 sekundi rdečega in svetlo modrega vzorca
- 1 sekunda modrega in rumenega vzorca

[3]

Informacije so shranjene v dveh blokih, oba bloka imata uvod, ki ustreza rdečemu in svetlo modremu vzorcu, dejanski program in podatki pa ustrezajo modremu in rumenemu vzorcu. Prvi blok je predhodni in vsebuje ime ter različne druge informacije o programu, drugi blok pa je sam program skupaj z vsemi prisotnimi spremenljivkami. Bel del med njima je le vrzel. [3]

5.5 Zvok

Delovanje in toni

ZX Spectrum lahko proizvaja neskončno raznolike zvoke, pri čemer lahko določimo tako frekvenco kot trajanje tona z ukazom BEEP. [3]

Ukazu podamo dolžino izvajanja tona v sekundah in višino tona v poltonih nad srednjim C. Osrednja frekvenca za ukaz BEEP je srednji C, lahko pa pridobimo katerikoli polton nad ali pod to osrednjo frekvenco. [3]

Str. 101 - prikaz tonov na klavirski tipkovnici [2]

Ker je v računalniku samo en zvočnik, lahko igrate samo en ton naenkrat. Proizveden zvočni signal ne pride le do zvočnika, temveč istočasno tudi do MIC in EAR **sockets**. [2]

Na zvočnik lahko poleg z ukazom BEEP preko ROMa vplivamo tudi z ukazi OUT skozi V/I vrata 0xFE. Bita 4 v zlogu na vratih določa stanje zvočnika (0 pomeni izključen, 1 pomeni vključen) - frekvenco pa določamo s hitrostjo menjav med stanjema. [1]

5.6 Implementacija v emulatorju

Poglavje 6

Evalvacija emulatorja

Prikaz delovanja par programov in iger

Primerjava z obstoječimi emulatorji - hitrost, natančnost

Poglavje 7

Zaključek

Evalvacija uspešnosti diplomskega dela (na podlagi ciljev iz uvoda)

Možne izboljšave

Boljša emulacija časovnih ciklov s strani pomnilnika, procesorja in ULA za boljšo sinhronizacijo in emulacijo strojnih napak dejanskega Spectruma: - 1. Memory contention - fetching instructions and other general memory reads/writes - I/O contention - Port 0xFE (ULA port) contention based on address bus high byte - Snow effect

Literatura

[1] Jaro Lajovic. *Strojni jezik za procesor Z80*. Ljubljana: Mladinska knjiga, 1985.

[2] Steven Vickers in Robin Bradbeer. *ZX Spectrum BASIC Programming, Third Edition*. Cambridge: Sinclair Research Limited, 1983.

[3] Steven Vickers in Robin Bradbeer. *ZX Spectrum Introduction, First Edition*. Cambridge: Sinclair Research Limited, 1982.

[4] Zilog. *Z80 CPU User manual*. 2016. URL: `https://www.zilog.com/docs/z80/um0080.pdf` (pridobljeno 1. 2. 2026).